

```
public class DatabaseService {  
  
    private String connectionString;  
  
    void processData(JSON data) {  
        ...  
  
        // Optimization complete  
  
        // Optimization complete  
    }  
  
    try {  
        ...  
        SELECT * FROM users;  
        ...  
  
        // Optimization complete  
        Resource = AR(JSON data);  
        ...  
    }  
}
```



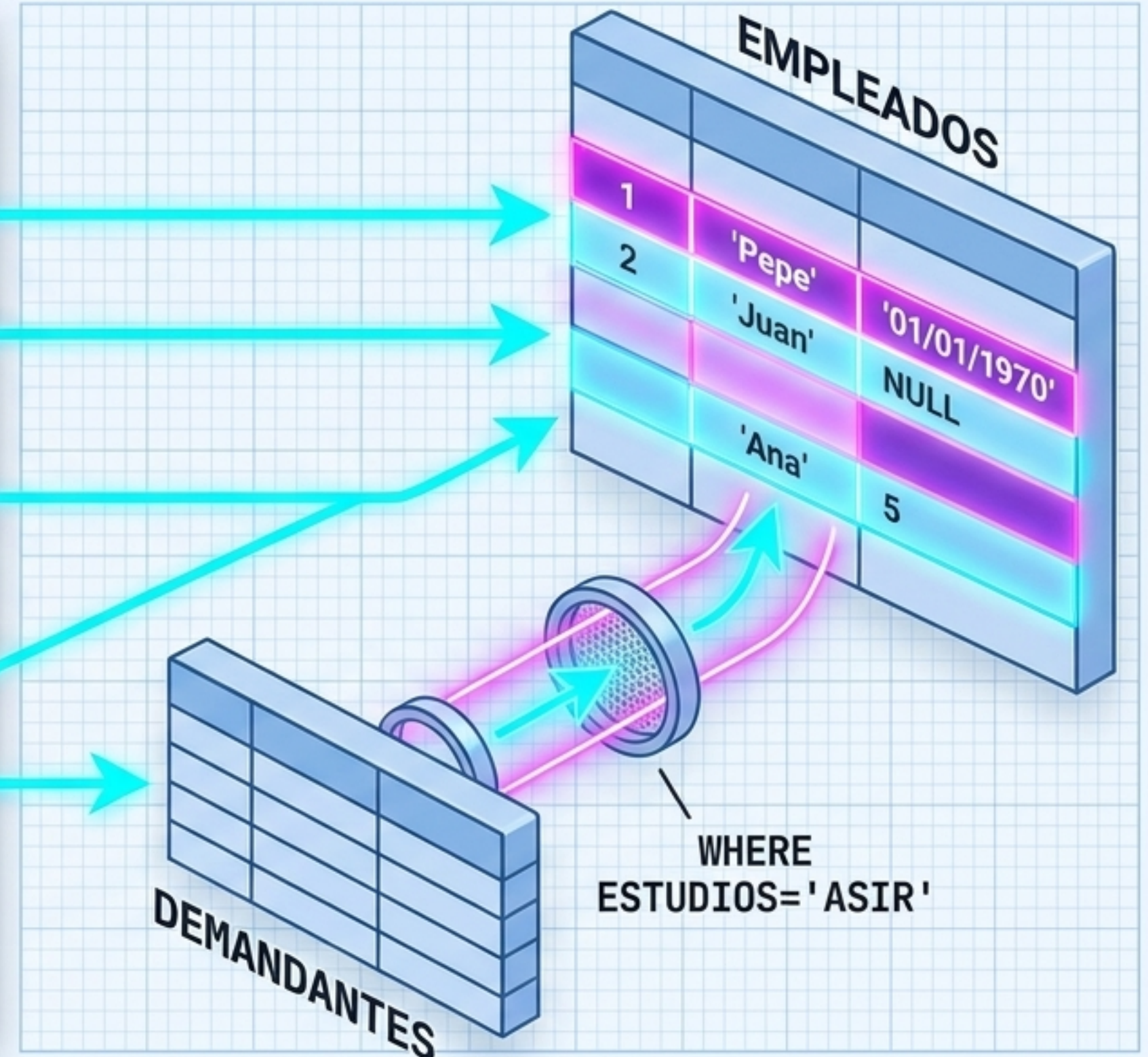
Arquitectura y Tratamiento de Datos: Nivel Maestro

Un manual visual desde la inserción básica (DML) hasta el rendimiento máximo, la manipulación de JSON/XML y la optimización de motores SQL.

El Origen de los Datos: Inserción Directa y Dinámica

```
INSERT INTO EMPLEADOS  
VALUES (1, 'Pepe', '01/01/1970');  
INSERT INTO EMPLEADOS  
VALUES (2, 'Juan', NULL);  
INSERT INTO EMPLEADOS(NOMBRE, ID_EMP)  
VALUES ('Ana', 5);
```

```
INSERT INTO EMPLEADOS  
SELECT ID_NUM, NOMBRE, NACIMIENTO  
FROM DEMANDANTES  
WHERE ESTUDIOS='ASIR';
```



Mantenimiento del Ciclo de Vida: Modificación y Poda

```
UPDATE EMPLEADOS
SET NOMBRE=INITCAP(NOMBRE), LOCALIDAD='Madrid'
WHERE FECHA_NAC >= '01/12/1980';

UPDATE EMPLEADOS
SET salario=salario*1.15
WHERE id_seccion = (SELECT id_seccion FROM secciones
                    WHERE nom_seccion='Productivo');
```



```
-- Se borran los empleados de Madrid
DELETE EMPLEADOS
WHERE LOCALIDAD='Madrid';

-- Se borran empleados de Ciudad Real
-- cuya fecha de nacimiento sea anterior a 1980
DELETE EMPLEADOS
WHERE FECHA_NAC < '01/01/1980' AND LOCALIDAD = 'C.Real';
```



Atención: Sin la cláusula WHERE, la operación vaciará la tabla entera.

La Matriz de Integridad Referencial

Acción	Metáfora	Resultado en la Tabla Hija
RESTRICT		Impide que se elimine el registro principal si existen dependencias.
CASCADE		Elimina automáticamente todos los registros vinculados.
SET NULL		Establece como nula la referencia en los registros hijos.
SET DEFAULT		Restaura un valor predeterminado en el campo.

ORA-00001: restricción única violada

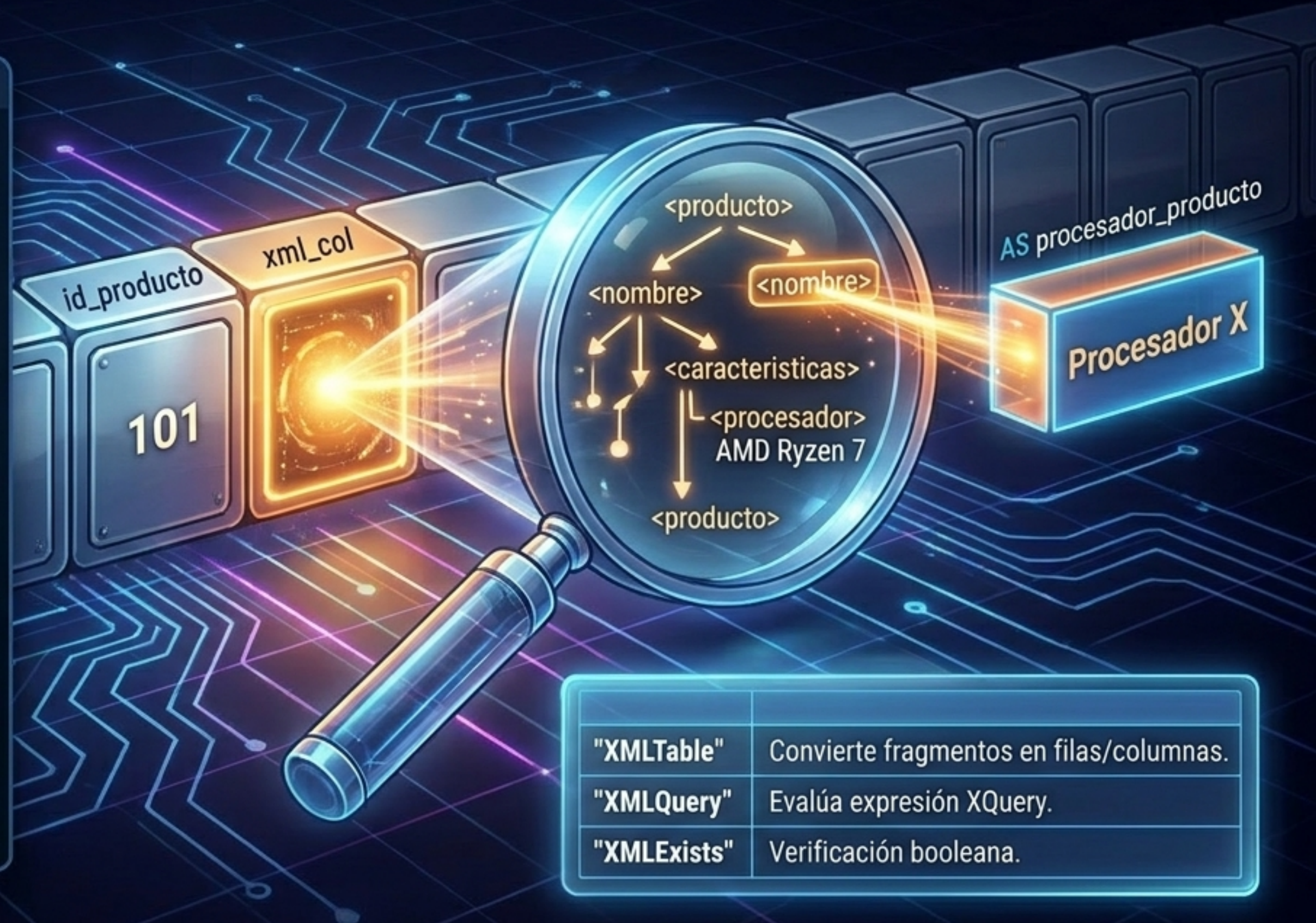
Conciencia del Sistema: El Diccionario de Datos

```
SELECT tables.table_name,  
       columns.column_name,  
       columns.data_type,  
       columns.data_length  
)  
FROM user_tables tables  
INNER JOIN user_tab_columns  
       columns ON  
       tables.table_name =  
       columns.table_name;  
ORDER BY TABLE_NAME ASC,  
         COLUMN_NAME ASC;
```



Rompiendo la Cuadrícula: Almacenamiento XML

```
CREATE TABLE ficha_producto (  
  id_producto NUMBER,  
  xml_col XMLType  
);  
  
SELECT EXTRACTVALUE(xml_col,  
  '/producto/nombre/text()')  
  AS procesador_producto  
FROM ficha_producto  
WHERE XMLExists('/producto/  
  características/procesador  
  [text()="AMD Ryzen 7"]'  
    PASSING xml_col);
```



Ingesta de Datos Modernos: Integración con JSON

```
CREATE TABLE datos_producto (  
  id_producto NUMBER,  
  json_col CLOB CHECK (json_col IS JSON)  
);
```

```
SELECT JSON_VALUE(json_col,  
  '$.producto.nombre')  
  AS nombre_producto  
FROM datos_producto  
WHERE id_producto = 1;
```

```
{  
  "producto": {  
    "nombre": "Laptop Gaming X1",  
    "caracteristicas": {  
      "procesador": "AMD Ryzen 7",  
      "ram": "32GB"  
    }  
  }  
}
```

procesador_producto

AMD Ryzen 7

"JSON_VALUE":	"Extrae un valor escalar."
"JSON_QUERY":	"Extrae un objeto/array."
"JSON_TABLE":	"Convierte a filas/columnas SQL."

Transformación Dimensional: PIVOT y UNPIVOT

AÑO	TRIMESTRE	VENTAS
2022	1	1000
2022	2	1500
2022	3	2000
2022	4	2000
2022	1	2500
2022	1	1100
2023	1	1600
2023	2	2100
2023	3	2600
2023	4	

PIVOT

```
SELECT *  
FROM  
  (SELECT año, trimestre, ventas  
   FROM ventas)  
PIVOT (SUM(ventas) FOR trimestre IN  
       (1 AS Q1, 2 AS Q2,  
        AS Q2, 3 AS Q3, 4 AS Q4))  
ORDER BY año;
```

UNPIVOT

```
SELECT año, trimestre, ventas  
FROM ventasQ  
UNPIVOT (ventas FOR trimestre IN  
        (Q1 AS 1, Q2 AS 2, Q3 AS 3, Q4 AS 4))  
ORDER BY año, trimestre;
```

AÑO	TRIMESTRE	VENTAS
2022	1	1000
2022	2	1500
2022	2	1500
2022	3	2000
2022	4	2500
2023	1	1100
2023	2	1600
2023	3	2100
2023	4	2600

El Espectro de Agregación Avanzada

ROLLUP



```
GROUP BY ROLLUP  
  (departamento, producto)
```

Genera subtotales jerárquicos
y un gran total.

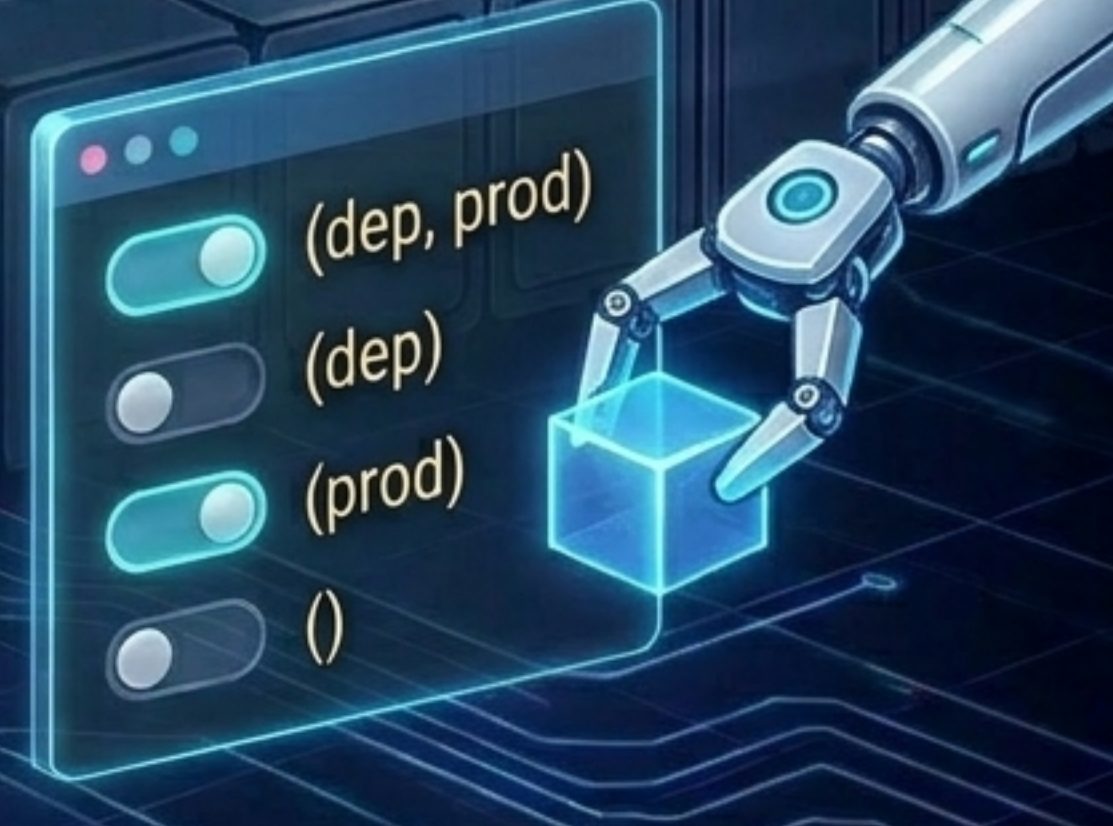
CUBE



```
GROUP BY CUBE  
  (departamento, producto)
```

Genera subtotales para todas las
permutaciones y cruces posibles.

GROUPING SETS



```
GROUP BY GROUPING SETS  
  ((dep, prod), (dep), (prod), ())
```

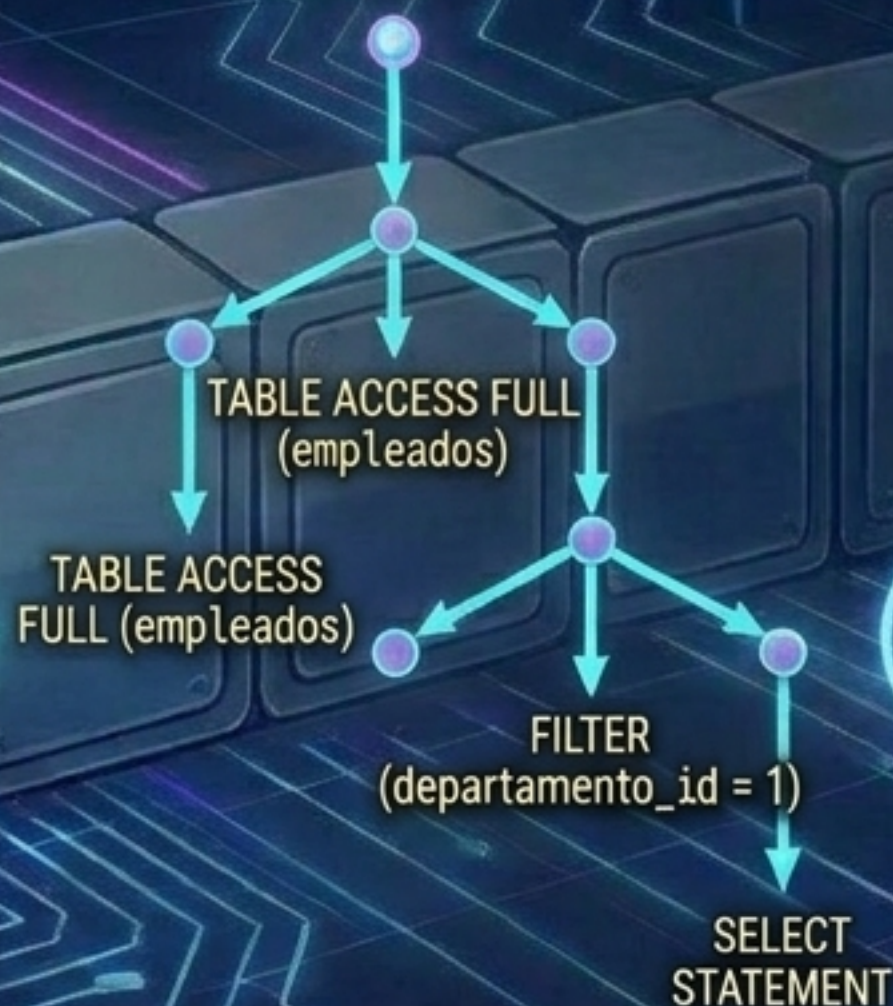
Flexibilidad extrema: calcula únicamente
las agrupaciones explícitamente solicitadas.

El Cerebro del Motor: Planes de Ejecución y Sugerencias

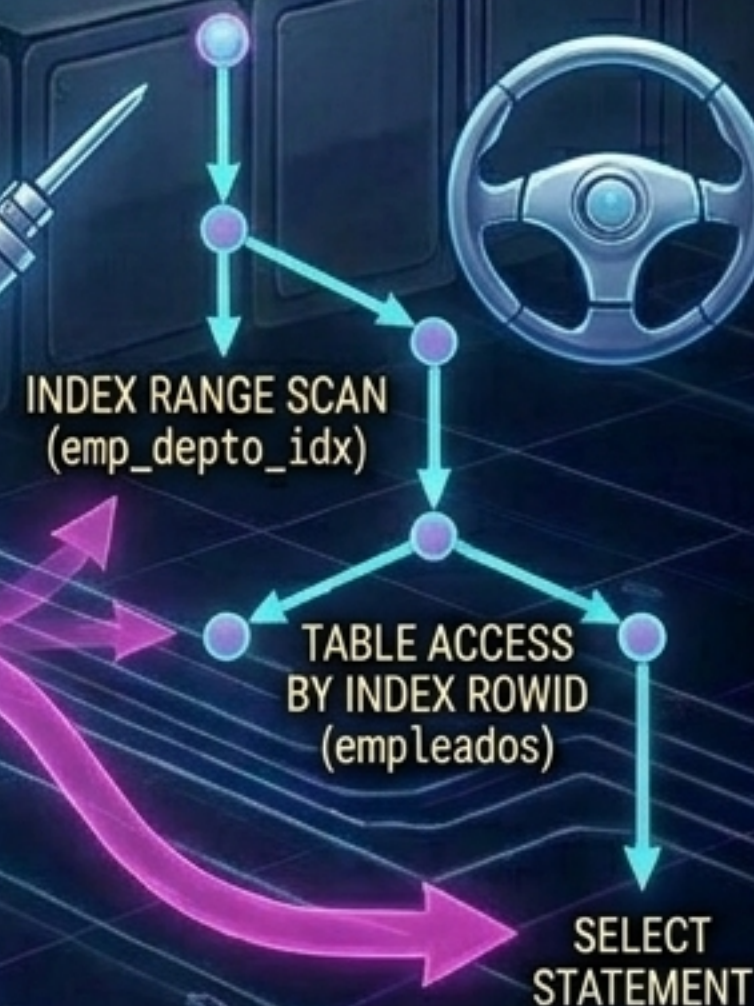
Motor de la Base de Datos

```
SELECT *  
FROM empleados  
WHERE departamento_id = 1;
```

EXPLAIN PLAN



Sugerencias (Hints)



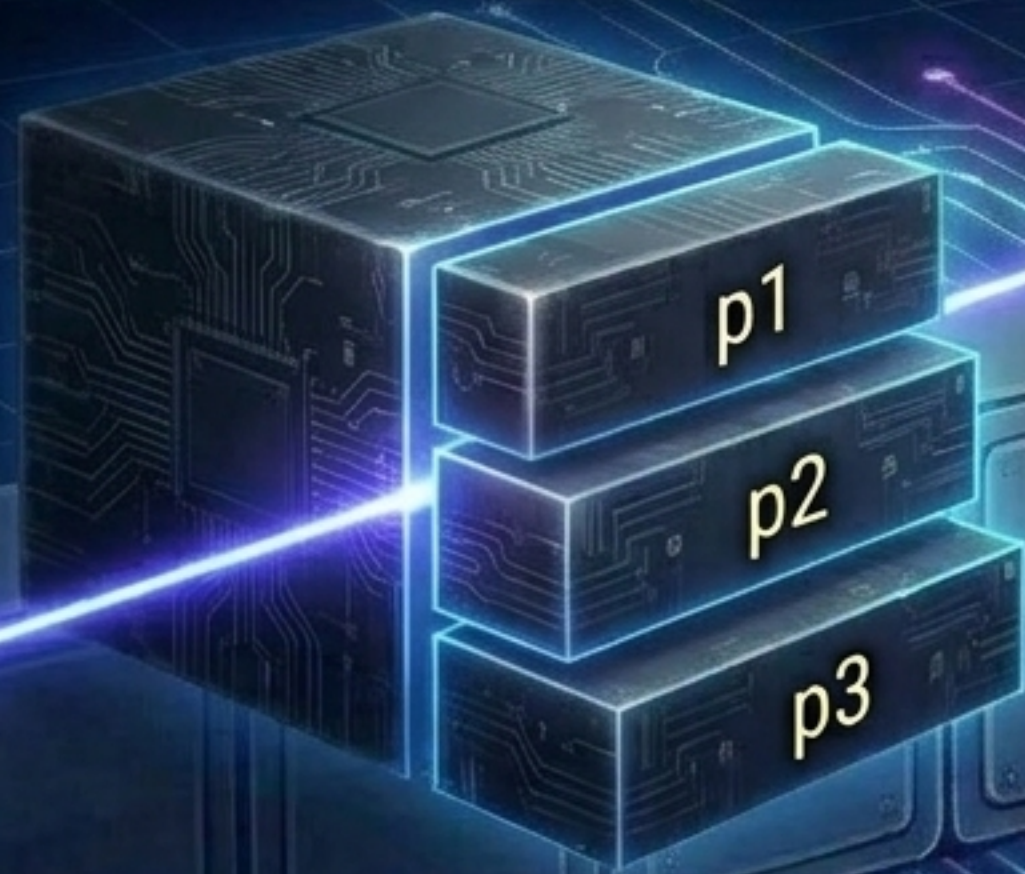
Motor de la Base de Datos

```
EXPLAIN PLAN FOR  
SELECT * FROM empleados ...;  
SELECT * FROM TABLE(DBMS_XPLAN.DISPLAY);
```

```
SELECT /*+ INDEX(empleados emp_depto_idx) */  
* FROM empleados ...;
```


Escalabilidad Física: Particiones y Vistas Materializadas

PARTICIONAMIENTO



```
PARTITION BY RANGE (fecha_contratacion) (  
  PARTITION p1 VALUES LESS THAN (TO_DATE('2022-01-01', 'YYYY-MM-DD')),  
  PARTITION p2 VALUES LESS THAN (TO_DATE('2023-01-01', 'YYYY-MM-DD')),  
  PARTITION p3 VALUES LESS THAN (MAXVALUE)  
);
```

Divide tablas gigantes en partes manejables, mejorando el rendimiento por rango.

VISTAS MATERIALIZADAS



```
CREATE MATERIALIZED VIEW emp_salario_depto  
AS  
  SELECT departamento_id, AVG(salario) avg_salario  
  FROM empleados  
  GROUP BY departamento_id;
```

Almacena físicamente los resultados de consultas complejas; congelando el cálculo para accesos instantáneos.

Eficiencia de Ejecución: Variables Enlazadas

Evita la recompilación masiva. Las variables enlazadas permiten que el motor trate consultas con diferentes valores como una estructura idéntica, reutilizando el Perfil SQL y ahorrando tiempo de procesamiento.

CONSULTA CON VARIABLE ENLAZADA

```
SELECT * FROM empleados  
WHERE departamento_id = :dept_id;
```

Asegura que el mismo Plan de Ejecución se utilice repetidamente, independientemente del valor específico de :dept_id.

El Blueprint del Arquitecto: Mapa de Tratamiento de Datos

1. DML & Integridad

Inserción,
Modificación,
Eliminación.

Restrict Cascade Null Default

2. Diccionario de Datos

USER_
(Actual)

ALL_
(Permitido)

DBA_
(Global)

SQL
Arquitectura
Nivel Maestro

3. Extensiones Estructurales



XML (Árboles)



JSON (Objetos/Arrays)

4. Transformación

PIVOT/UNPIVOT

ROLLUP

CUBE

GROUPING SETS

5. Optimización Física

Planes de
Ejecución

Hints

Particiones

Vistas
Materializadas

Variables
Enlazadas

Diseñado para la eficiencia, construido para la escala.